RISC-V 指令集 CPU 设计

——基于紫光同创 FPGA 的五级流水处理器与 INT8 边缘 AI 加速单元

摘　要

本作品面向紫光同创 FPGA 平台，设计并实现了一款 RV32I 指令集的 RISC-V 处理器，以及配套的存储层次、外设子系统与 INT8 边缘 AI 加速单元。处理器采用五级流水结构（IF/ID/EX/MEM/WB），在基础整数指令集之上扩展了 Zicsr 控制状态寄存器与 M 扩展乘除法，并实现了两路组相联的指令/数据 Cache、BTB 加两位饱和计数器的动态分支预测，以及包含外部中断、向量 mtvec 与非对齐访存精确异常的完整机器模式异常处理机制。

存储子系统是本设计的一个重点：指令与数据主存采用 4096×128 位的宽存储结构，Cache 缺失时以整行突发方式一次读回 16 字节，缺失代价由 5 拍降到 3 拍；在综合阶段，整行读曾被 4 个读端口破坏 BRAM 推断（LUT 由 6 036 膨胀到 57 495），我们把存储器本身改为宽字结构并用 16 路字节使能完成 32 位写，最终恢复为 30 个 BRAM 的正常映射。

SoC 集成 UART、GPIO、64 位定时器与一个 memory-mapped 的 AI 加速器：加速器内含 8 KB 本地缓冲与 4 路 INT8 乘累加引擎，支持全连接、ReLU 与 Argmax 三类算子；配套的 MNIST 手写数字识别模型（196-32-10 MLP）经训练与量化后部署到加速器，20 张测试图识别正确 19 张（95%），与 PC 端 INT8 仿真精度 96.19% 相当。

验证方面，全部 RTL 在 Icarus Verilog 与 Vivado xsim 两套仿真器上交叉验证，15 项自检测试全部通过；CoreMark 2K performance run 的三项 CRC 与官方值一致，计时区 357 397 拍，折算约 2.80 CoreMark/MHz；全 SoC 综合约 10 919 LUT、3 726 个触发器、28 个 BRAM 与 20 个 DSP。受板卡尚未到位的影响，目前完成的是设计、仿真与综合评估，上板验证与实物照片将在板卡到位后补充。

第一部分　作品概述

1.1 创意来源

RISC-V 作为开放指令集，近年来在国产处理器生态中发展迅速；而紫光同创等国产 FPGA 提供了自主可控的可编程逻辑平台。把二者结合起来——在国产 FPGA 上用 Verilog 实现一颗真正能跑分、能跑应用的 RISC-V 处理器——既有技术挑战，也有明确的工程价值。

我们的出发点不是"堆一个能通指令的核"，而是补齐一颗小处理器真正缺的东西：取指与访存带宽、分支代价、异常与中断的完整性，以及一颗可编程器件上"能演示"的智能应用。因此在完成

RV32I 五级流水之后，我们继续做了两路 Cache 与整行突发、BTB 动态预测、M 扩展、以及一块 INT8 矩阵-向量加速器，并用 CoreMark 与 MNIST 两个真实负载检验设计，而不是只跑通几个手写指令序列。

在缺少实体板卡的阶段，我们把"可验证"放在第一位：自研汇编器摆脱对交叉工具链的依赖，自建自检平台与 15 项回归用例，再用开源 RISC-V GCC 编译 CoreMark 与 MNIST 推理程序，在两套仿真器上交叉验证。这样板卡到位时，工程只需要补板级顶层与引脚约束即可上板。

1.2 应用场景

（1）教学与竞赛：完整展示从指令译码、流水线冒险、Cache 到 AI 加速的处理器设计全流程，配套自检用例与文档可直接作为实验平台。

（2）边缘智能终端：加速器面向 batch=1 的 INT8 推理场景，可用于手写数字识别、简单字符/形状分类、传感器信号分类等低功耗边缘任务。

（3）工业控制与人机交互：内核带 UART/GPIO/定时器与完整中断机制，可作为小型控制器的主控核，用于电机控制、面板交互或数据采集。

（4）国产化替代验证：在国产 FPGA 上验证 RISC-V 软硬件栈（工具链、裸机运行时、基准测试、轻量模型）的可用性，为后续产品化积累数据。

1.3 主要功能特点

① 处理器内核：RV32I 全 37 条指令 + Zicsr + M 扩展（8 条乘除），五级流水，全转发、load-use 停顿、WB→ID 旁路与 CSR 互锁，分支在 EX 级解析并由 BTB 预测。

② 存储层次：2 KB 两路组相联 I/D Cache（伪 LRU、16 B 行、写穿），64 KB + 64 KB 宽字主存（4096×128 位，BRAM），缺失整行突发填充。

③ 异常与中断：非法指令、ECALL/EBREAK、指令/数据地址非对齐、机器定时器中断、机器外部中断，支持直接/向量两种 mtvec，mret 恢复现场，WFI 按 NOP 实现。

④ 外设与 SoC：UART 8N1、GPIO、64 位定时器、测试退出寄存器与软件中断寄存器，地址译码按 4 KB 页区分，外设访问绕开 D-Cache。

⑤ AI 加速：8 KB 本地缓冲 + 4 路 INT8 MAC，支持 FC/ReLU/Argmax，memory-mapped 访问，配套量化流程与 MNIST 推理演示。

⑥ 软件生态：自研汇编器与镜像拆分工具、裸机 C 运行时（crt0/链接脚本/UART 输出）、CoreMark 与 MNIST 两套真实负载、一键回归脚本。

1.4 主要性能指标

指标	数值	说明
流水线	5 级	IF/ID/EX/MEM/WB，分支 EX 解析
工艺/平台	紫光同创 PGL22G（待定）	Verilog-2001，可综合
综合资源	10 919 LUT / 3 726 FF	含 AI，Artix-7 OOC 估算
存储资源	28 个 BRAM	2×64 KB 主存 + Cache
CoreMark	357 397 拍/迭代	CRC 与官方一致，≈2.80 CoreMark/MHz
MNIST	19/20（95%）	INT8 加速器实测
IPC	0.35～0.98	依负载，t08 接近 1.0

1.5 主要创新点

· 整行突发的存储层次：主存 128 位宽、Cache 按行存储，缺失 3 拍完成，同时保持 BRAM 可推断。

· 低开销动态分支预测：64 项 BTB + 两位计数器，预测正确零冲刷，t08 IPC 0.59→0.98。

· 乘法器下沉 MEM 级：把乘法从 EX 长路径移出，逻辑延迟 10.28→4.08 ns，WNS −6.06→−2.15 ns。

· 面向 batch=1 的轻量 AI 加速：8 KB 缓冲 + 4 路 INT8 MAC，配校准集定点量化，MNIST 95%。

· 可复现的验证体系：自研汇编器 + 15 项自检 + 双仿真器 + CoreMark/MNIST 真实负载。

1.6 设计流程

需求拆解 → 微架构设计（流水线/存储/中断）→ RTL 实现 → 自研汇编器与自检平台 → 逐模块回归 → CoreMark/MNIST 真实负载 → Vivado 综合与资源/时序评估 → 板级顶层与 PDS 约束（待板卡）。

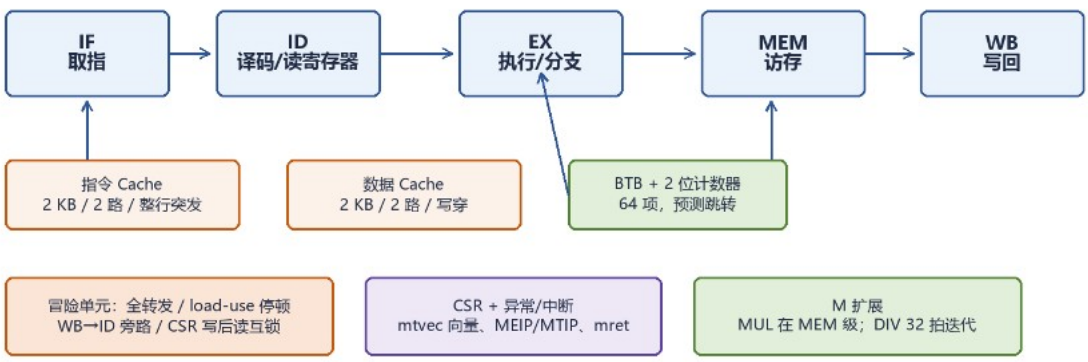


图 1 五级流水线结构

第二部分　系统组成及功能说明

2.1 整体介绍

系统由处理器内核、存储层次、外设子系统与 AI 加速器四部分组成，全部集成在 soc_top 中，对外只暴露时钟、复位、UART、GPIO 等板级信号。内核通过独立的取指口与数据口分别连接 I-Cache 与 D-Cache；Cache 缺失时经整行突发访问 128 位宽的 BRAM 主存；外设与加速器通过数据总线上的地址译码访问，不经过 Cache。

2.1.1 系统组成框图

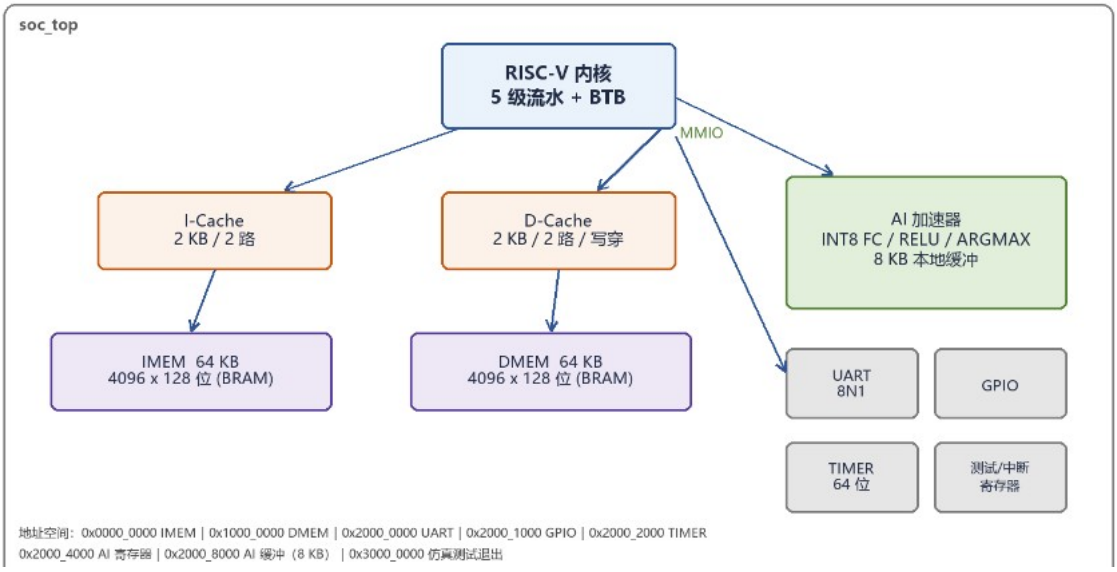


图 2 SoC 总体结构与地址空间

图中各模块职责如下：RISC-V 内核负责取指、译码、执行、访存与写回，并内含 BTB 与 CSR；I-Cache/D-Cache 为两路组相联、各 2 KB，负责把 64 KB 主存的带宽"放大"到每拍可取指/访存；AI 加速器挂在自己的寄存器窗口与 8 KB 缓冲上，独立于 Cache 层次；UART/GPIO/TIMER 与测试、中断寄存器构成外设区。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

目标平台为紫光同创 PGL22G 系列开发板（具体型号待板卡到位后确认）。按综合评估，全 SoC 约 10.9K LUT、3.7K 触发器、28 个 BRAM、20 个 DSP，对 PGL22G 的逻辑资源占用约一半，属于可容纳但偏紧的规模；若板卡资源更紧张，可将 AI 缓冲改为同步读 BRAM、或将加速器阵列由 4 路降为 2 路。

板级顶层 fpga_top 已完成：包含复位同步、时钟直连（预留 PLL 位置）、UART 收发、LED 状态指示与按键输入，并将 SoC 的测试通过/失败状态映射到 LED。约束方面提供了 PDS 的引脚约束模板与 SDC 时序模板，板卡确定后按原理图填写即可。

2.2.2 机械设计介绍

本作品为纯数字逻辑设计，使用现成开发板，不涉及机械结构设计。

2.2.3 电路设计介绍

本作品不自行设计 PCB，直接使用开发板板载资源（时钟、复位按键、UART 接口、LED、按键）。与硬件的接口仅为 FPGA 引脚，约束模板见代码目录 constraints/pango_pins_template.fdc；时序约束模板见 constraints/timing_template.sdc。

2.3 软件系统介绍

2.3.1 软件整体介绍

软件部分包含四块：① 自研 RV32I 汇编器（支持伪指令、数据段、符号与重定位），用于生成免工具链的测试镜像；② 裸机 C 运行时（crt0、链接脚本、UART 输出、mcycle 计时），支撑 CoreMark 与 MNIST 程序；③ PC 端训练与量化脚本（numpy 训练 MLP、per-tensor 量化、校准集定点缩放、导出 C 头文件）；④ 仿真与回归脚本（Icarus Verilog 与 Vivado xsim 两套流程）。

2.3.2 软件模块介绍

· rvasm.py：两遍扫描，第一遍定地址与符号，第二遍编码；支持 .text/.data、.word/.byte/.align、li/la/mv/bnez 等伪指令与 CSR 助记符。

· hexsplit.py：把 GCC 生成的 Intel HEX 按地址拆成 imem.hex 与 dmem.hex（128 位整行格式）。

· train_mnist.py：MNIST 下采样到 14×14，训练 196-32-10 MLP，量化权重与偏置，按校准集确定激活缩放系数，模拟 INT8 推理并导出头文件。

· mnist.c：把权重与图像写入 AI 缓冲，按 8 组 + 3 组调用 FC，CPU 侧完成偏置、ReLU 与定点缩放，最后用硬件 Argmax 得到类别并通过 UART 输出。

· run.ps1 / run_xsim.ps1：一键汇编、编译、仿真、汇总；支持单测、波形、跟踪与统计。

第三部分　完成情况及性能参数

3.1 整体介绍

目前已完成处理器内核、存储层次、外设、AI 加速器的 RTL 设计，完成自检平台与 15 项回归、CoreMark 与 MNIST 两套真实负载的仿真验证，并完成 Vivado 综合的资源与时序评估。

受板卡尚未到位影响，本部分暂缺实物照片（整机、45° 全貌、接口特写等），将在租借板卡后补充：整体实物图、开发板关键角度照片、上板运行 CoreMark/MNIST 的现场照片。本版本以仿真结果作为完成情况的证据。

3.2 工程成果

3.2.1 机械成果

无（使用现成开发板）。

3.2.2 电路成果

无自绘 PCB；提供 PDS 引脚与时序约束模板，待板卡确定后填写。

3.2.3 软件成果

代码按功能分层组织：rtl/core（内核）、rtl/mem（Cache 与主存）、rtl/ai（加速器）、rtl/periph（外设）、rtl/soc（顶层）、tb（测试平台）、sw（汇编测试、C 程序、工具链脚本）。主要文件与行数见第四部分"文档情况"。

3.3 测试成果

3.3.1 功能测试

测试	覆盖点	周期	结果
t01_alu	ALU/立即数/移位/比较/LUI/AUIPC	216	通过
t02_branch	6 种分支、清空错误路径、循环	258	通过
t03_mem	字节/半字/字、符号扩展、load-use	175	通过
t04_jump	JAL/JALR 链接值、嵌套调用	174	通过
t05_csr_trap	CSR、ecall/ebreak/非法指令、mret	171	通过
t06_sort	冒泡排序集成测试	580	通过
t07_hazard	依赖链、旁路、互锁	137	通过
t08_irq	定时器中断与 mret 恢复	6 499	通过
t09_periph	GPIO、定时器、UART 状态	245	通过
t10_cache	4 KB 顺序读写 + 热点访问	15 260	通过
t11_conflict	2 KB 间隔数组交替访问	444	通过

测试	覆盖点	周期	结果
t12_muldiv	乘除边界（除零/溢出/符号）	532	通过
t13_trap_ext	向量 mtvec、外部中断、非对齐访存	180	通过
t14_ai	加速器 FC/RELU/ARGMAX	206	通过

3.3.2 性能测试

项目	结果
CoreMark 2K 计时区	357 397 拍（CRC：0xe9f5 / 0xe714 / 0x1fd7 / 0x8e3a）
CoreMark 折算	≈2.80 CoreMark/MHz
MNIST 识别	19/20（95%），整程序 170 691 拍
综合资源（含 AI）	10 919 LUT / 3 726 FF / 28 BRAM / 20 DSP
时序（Artix-7 OOC）	WNS −3.10 ns，估算 ≈76 MHz（未布局布线）

3.3.3 与同类设计对比（定性）

与常见的教学级 RISC-V 核相比，本设计在三个方面更完整：一是存储层次包含两路 Cache 与整行突发，而不是直接连组合读存储器；二是具备动态分支预测与外部中断、向量入口等接近实用的控制逻辑；三是带有一块可编程的 AI 加速器并跑通了真实模型，而不是仅停留在指令集层面。

第四部分　总结

4.1 可扩展之处

（1）频率：把乘法器改为带流水寄存器的 DSP 结构，目标 100 MHz；（2）Cache：实现关键块优先与非阻塞填充，缺失代价进一步降到 2 拍；（3）AI：把加速器扩展为 8×8 脉动阵列并支持卷积/池化，可运行小型 CNN；（4）生态：补齐 A 扩展与轻量 RTOS 支持，接入更多外设（SPI/I2C、摄像头）。

4.2 文档情况

本技术文档按组委会模板组织，共五部分：作品概述（创意来源、应用场景、功能特点、性能指标、创新点、设计流程）、系统组成及功能说明（整体框图、硬件与软件介绍）、完成情况及性能参数（工程成果与测试结果）、总结（可扩展性与文档情况）、参考文献。

配套工程文档位于代码目录 doc/ 下：《01_架构设计.md》描述流水线、Cache、BTB、异常中断、AI 加速器的结构与参数；《02_验证报告.md》记录 15 项回归、CoreMark、MNIST、综合资源与调试过程中发现的问题；《03_后续路线.md》给出频率、Cache、AI 与板级工作的后续计划。README.md 提供快速上手与复现命令。

代码规模：RTL 约 20 个 Verilog 文件（内核 8 个、存储 4 个、AI 1 个、外设 4 个、顶层 2 个），测试平台 1 个，汇编测试 15 个，C 程序 2 个，Python 工具 3 个，PowerShell 脚本 4 个。全部源码与仿真日志随作品提交。

第五部分　参考文献

[1] Andrew Waterman, Krste Asanović. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA. 2019.

[2] Andrew Waterman, Krste Asanović. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture. 2021.

[3] David A. Patterson, John L. Hennessy. Computer Organization and Design: RISC-V Edition. Morgan Kaufmann, 2017.

[4] Sarah L. Harris, David Money Harris. Digital Design and Computer Architecture: RISC-V Edition. 2021.

[5] EEMBC. CoreMark: An Industry Standard Benchmark for Embedded Processors. 2023.

[6] Yann LeCun, Corinna Cortes, Christopher J.C. Burges. The MNIST Database of Handwritten Digits.

[7] 紫光同创. PGL22G 器件手册与 PDS 开发工具用户指南.

[8] 胡伟武等. 计算机体系结构（RISC-V 版）. 清华大学出版社, 2022.

[9] Jacob Benesty, et al. 定点量化与神经网络压缩相关综述（IEEE Signal Processing Magazine, 2020）.

[10] 赛题一：基于紫光同创 RISC-V 指令集 CPU 设计. 2025 紫光同创杯 FPGA 创新大赛.